

PROMETHEUS CERTIFIED ASSOCIATE · SCALING & LONG-TERM
STORAGE MODULE

Prometheus Scaling & Long-Term Storage

A complete, step-by-step, easy-to-remember guide — covering TSDB internals, retention tuning, capacity planning, remote read/write, Thanos, vertical & horizontal scaling, hashmod sharding, and federation with recording rules.

One Prometheus overloaded

→

Shard / Scale it

→

Federate the summary

→

Offload to Thanos

Table of Contents

1. Long-Term Storage & TSDB	blocks · retention · capacity
2. Remote Read & Remote Write	streaming to external stores
3. Thanos	Sidecar · Store · Query
4. Scaling Prometheus	vertical vs horizontal
5. Horizontal Sharding	hashmod relabeling
6. Reducing Load: Intervals & Sample Limits	quick wins
7. Federation	global Prometheus, safely
8. Final Cheat Sheet	everything on one page

i How to use this guide: Each section opens with a short "memory hook" box (yellow, dashed border) — read that first, it gives you a one-line trick to recall the whole concept before you dive into the technical detail.

1. Long-Term Storage & TSDB

Foundations

Prometheus writes collected samples into a block-based **TSDB** (time-series database) on local disk. New blocks are created every **two hours** by default. The storage directory is configurable with `--storage.tsdb.path`.

M MEMORY HOOK

Think of the TSDB directory like a filing cabinet that gets a **new drawer every 2 hours**. Each drawer (block) has a label (`meta.json`), the actual papers (`chunks`), an index card catalog (`index`), and a "shredded" pile for deleted records (`tombstones`).

1.1 Anatomy of the data directory

```
./data
├── 01BKGV7JBM69T2G1BG6BGM6KB12
│   └── meta.json
├── 01BKGV7JC0RYB86MACW02A2PJDJ
│   ├── chunks
│   │   └── 000001
│   ├── tombstones
│   ├── index
│   ├── meta.json
│   ├── chunks_head
│   │   └── 000001
│   └── wal
│       ├── 00000002
│       ├── checkpoint.000001
│       └── 00000000
```

Component	Purpose
<code>chunks</code>	Compressed time-series samples
<code>index</code>	Label & metric-name indexes for fast lookups inside the block
<code>meta.json</code>	Block metadata (time ranges, version, etc.)
<code>tombstones</code>	Records of deleted series
<code>wal</code> / <code>chunks_head</code>	Write-ahead log & in-progress (not-yet-blocked) chunk data

1.2 Retention: defaults and flags

By default, Prometheus keeps roughly **15 days** of data or up to **10 GB** — whichever limit is hit first, the oldest blocks get deleted.

```
# limit by age
--storage.tsdb.retention.time=15d

# limit by disk size
--storage.tsdb.retention.size=10G
```

Flag	Purpose	Default	Example
<code>--storage.tsdb.path</code>	Path for the TSDB	<code>/prometheus</code>	<code>--storage.tsdb.path=/data</code>
<code>--storage.tsdb.retention.time</code>	Max age of data kept	<code>15d</code>	<code>--storage.tsdb.retention.time=30d</code>
<code>--storage.tsdb.retention.size</code>	Max storage size kept	<code>10G</code>	<code>--storage.tsdb.retention.size=50G</code>

! WARNING If retention flags don't match your real ingestion rate, Prometheus may delete data sooner than expected, or exhaust disk space. Always plan capacity and monitor disk usage.

1.3 Capacity planning formula

M MEMORY HOOK

"**T x S x B**" — Time (retention, in seconds) × Samples/sec ingested × Bytes per sample = disk you need.

disk required = retention time (seconds) × ingested samples per second × bytes per sample

- A compacted Prometheus sample is typically ~1-2 bytes, but real bytes/sample varies with label cardinality and compression.
- For accurate planning, measure your actual average samples/sec and bytes/sample under representative load — don't just trust the rule of thumb.

1.4 Why local-only storage doesn't scale

- Block storage gets expensive as retention and cardinality grow.
- Prometheus' TSDB writes to a **single local path** — it does not horizontally distribute blocks across multiple Prometheus instances.

These two limits make local-only retention impractical for multi-year storage — which is exactly the problem remote write and systems like Thanos solve (next sections).

2. Remote Read & Remote Write

Escaping Local Disk

M MEMORY HOOK

Write = Prometheus **pushes** samples out as they're scraped. **Read** = Prometheus **pulls** historical data back in when a query needs it. Same direction as the words suggest!

To avoid the cost and limits of local block storage, Prometheus supports:

- **remote_write** — stream samples to a remote system or long-term store
- **remote_read** — query historical data back from a remote store

This lets you push metrics into object storage (S3, Azure Blob) or scalable TSDB systems (InfluxDB, VictoriaMetrics, Thanos, Grafana Mimir).

```
remote_write:
- url: https://some-url.com/api/prom/push
  write_relabel_configs:
  - action: keep
    source_labels: [ "team" ]
    regex: 'frontend'

remote_read:
- url: http://some-url.com:1234/read
```

TIP `write_relabel_configs` can filter what gets sent remotely (as shown above, only the `frontend` team's series) — useful for reducing cardinality or routing specific metrics to specific remote systems.

2.1 Long-term storage systems

Systems like **Thanos** and **Grafana Mimir** (and TSDBs like VictoriaMetrics or InfluxDB) typically:

- Receive replicated data via `remote_write`, or ingest Prometheus blocks directly
- Store compacted data cheaply in object stores (S3, Azure Blob)
- Offer a queryable API that Prometheus or an external query layer can use

EXAM TIP For certification-level knowledge, you only need to know *that* Thanos and Grafana Mimir exist to provide cost-effective long-term storage — not their deep internals.

3. Thanos

Global View & Long-Term Storage

M MEMORY HOOK — "S-S-Q"

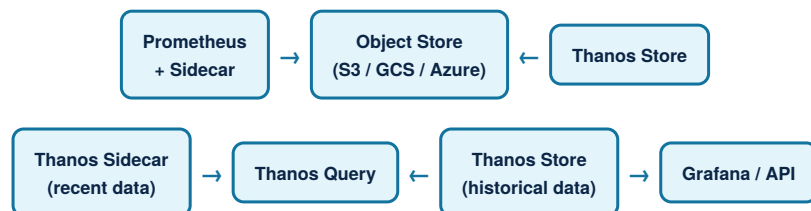
Sidecar (rides along with each Prometheus) → Store (reads old blocks from the object store) → Query (the one door you knock on for everything). Alphabetically it's almost "SSQ" — Sidecar Ships, Store Serves, Query Quizzes.

3.1 Why use Thanos with Prometheus?

- **Single pane of glass** — query many Prometheus instances through one Thanos Query endpoint instead of logging into each server individually.
- **HA deduplication** — when running Prometheus in HA (two servers scraping the same targets), Thanos deduplicates the duplicate series into one logical view.
- **Cost-effective long-term storage** — Thanos uploads TSDB blocks to object stores (S3, Azure Blob, GCS) so local disk doesn't grow unbounded.

3.2 Core components

Component	Responsibility	Typical deployment
Thanos Sidecar	Runs alongside each Prometheus; uploads TSDB blocks to object storage and serves recent local data over gRPC	Sidecar container in the same pod as Prometheus
Thanos Store	Reads historical TSDB blocks from object storage and serves them over gRPC	Stateless, accesses the object-store bucket
Thanos Query	PromQL-aware frontend; federates queries across sidecars + stores and deduplicates	Single (or HA) entry point for Grafana/API clients



Typical query flow: a user sends a PromQL query to Thanos Query, which fans it out to relevant sidecars (recent data) and to Thanos Store (historical blocks), then merges and deduplicates the results before returning them.

3.3 Compaction vs. Sidecar uploads

Prometheus writes TSDB blocks (every 2h) and periodically **compacts** small blocks into larger ones. But compaction can rewrite, merge, or delete blocks *while* the Thanos Sidecar is uploading them — causing upload races or failures.

To keep block layouts stable for uploads, many deployments disable Prometheus's automatic compaction by setting min and max block duration to the same value:

```
--storage.tsdb.min-block-duration=2h
--storage.tsdb.max-block-duration=2h
```

! WARNING Disabling compaction reduces Prometheus's on-disk optimizations and can affect local query performance or disk usage. Test in staging and monitor resource use before applying to production.

i EXAM TIP Focus on *purpose* (unified query, HA dedup, long-term storage) and the three component roles (Sidecar / Store / Query) — full internal architecture isn't required.

4. Scaling Prometheus

Vertical vs Horizontal

A single Prometheus server eventually hits limits: too many scrape targets slow it down, and every config change requires a restart (which means waiting for the next scrape interval to see fresh data again). The fix: run **multiple** Prometheus servers.

M MEMORY HOOK

Vertical scaling = **different departments**, each with their own manager (Prometheus A watches Service A, Prometheus B watches Service B). **Horizontal** scaling = **same department, split shift** (both managers watch Service A, but each covers half the shop floor).

4.1 Vertical scaling

Each Prometheus server is responsible for a **different set** of applications/services/devices. For example, Prometheus A monitors Services A+B, Prometheus B monitors Services C+D.

Benefits

- Cardinality considerations become visible to the team that owns each Prometheus instance.
- Teams feel empowered *and* accountable for monitoring their own services.
- Enables experimentation with rules and dashboards without fear of impacting other teams.

4.2 Horizontal scaling (sharding)

One very large scrape job (e.g. thousands of instances of Service A) is **split across multiple Prometheus servers** — this is called **sharding**. Server 1 covers half the instances, Server 2 covers the other half.

i KEY DISTINCTION Vertical scaling splits *different* services across servers. Horizontal scaling splits *one* service's many instances across servers. See Section 5 for the exact hashmod configuration.

5. Horizontal Sharding

hashmod Relabeling

M MEMORY HOOK

Think of `hashmod` like a **bouncer with a coin-flip machine** at each Prometheus server's door: every target's address gets hashed and modulo'd into a number 0..N-1. Each server only lets in targets whose number matches its own assigned shard index.

5.1 Key terms

Term	Meaning
<code>hashmod</code>	Relabel action: computes <code>hash(target) % modulus</code> , stores result in a label
Shard index	The integer (0..N-1) assigned to a given Prometheus instance
<code>__address__</code>	Label holding the target address (<code>host:port</code>)

5.2 The 3-step recipe

- 1 Hash each target's address into a temp label**
Use `action: hashmod` with `modulus` set to the total number of Prometheus instances (shards).
- 2 Keep only targets matching this instance's shard index**
Use `action: keep` with `regex` set to `0`, `1`, ... up to `modulus - 1`.
- 3 Repeat identically on every instance**
Same full target list, same `modulus` — only the `regex` shard index changes per instance.

5.3 Example: sharding across 2 Prometheus servers

Prometheus instance A (shard 0):

```
scrape_configs:
- job_name: "node"
  static_configs:
  - targets:
    - "192.168.64.8:9100"
    - "192.168.64.8:9101"
    - "192.168.64.10:9100"
    - "192.168.64.10:9101"
  relabel_configs:
    # Step 1: hash the __address__ into a temp label
    - source_labels: [__address__]
      action: hashmod
      modulus: 2
      target_label: __tmp_hashmod

    # Step 2: keep only targets mapping to shard 0
    - source_labels: [__tmp_hashmod]
      regex: "0"
      action: keep
```

Prometheus instance B (shard 1): identical, except `regex: "1"`.

Step	Action	Result
1	Compute <code>hashmod(__address__) % modulus</code>	Result (0..N-1) stored in <code>__tmp_hashmod</code>
2	<code>keep</code> where <code>__tmp_hashmod</code> matches this shard's <code>regex</code>	Only matching targets survive for that instance
3	Repeat on each instance with its own shard index	Targets partitioned across N Prometheus servers

! IMPORTANT Every Prometheus instance's static config must list **all** targets (not a pre-split subset) — the `keep` relabel is what actually does the filtering.

i TIP Set `modulus` to your number of shards. `hashmod` gives a near-uniform split with many targets, but distribution may be uneven with only a few. Changing the modulus later (adding/removing shards) reshuffles assignments — expect target movement.

5.4 Verification

- Check Prometheus's **Status** → **Targets** page on each instance to confirm it's scraping only its expected subset.

6. Reducing Load: Intervals & Sample Limits

Quick Wins

M MEMORY HOOK

Before you shard or add servers, try the cheap fixes first: **scrape less often**, and **cap runaway jobs**. Like turning down a faucet before calling a plumber.

6.1 Widen the scrape interval for slow / low-priority jobs

```
scrape_configs:
- job_name: 'slow-metrics-job'
  scrape_interval: 1m # instead of the default 15s
  static_configs:
    - targets: ['localhost:9100']
```

If you don't need second-by-second freshness, scraping less often directly reduces load.

6.2 Cap runaway scrapes with `sample_limit`

Acts like a safety fuse: if a single scrape returns more samples than the limit, the whole scrape is marked a **failure** and dropped — protecting Prometheus from a misconfigured target flooding it with metrics.

```
scrape_configs:
- job_name: 'example'
  static_configs:
    - targets: ['localhost:8080']
  sample_limit: 10000 # only ingest up to 10,000 samples per scrape
```

i TIP These two settings are the simplest levers — try them before reaching for vertical scaling, horizontal sharding, or federation.

7. Federation

Global Prometheus, Done Right

M MEMORY HOOK

Federation is a **"headline summary," not a photocopier**. The global Prometheus should read the *front-page digest* (aggregated recording-rule metrics) from each local Prometheus — never the entire raw newspaper (every instance-level metric).

7.1 The scenario

- Prometheus 1 scrapes nodes in Datacenter 1
- Prometheus 2 scrapes nodes in Datacenter 2
- A Global Prometheus federates from both via the `/federate` endpoint, giving a single pane of glass

7.2 Step 1 — local Prometheus servers scrape locally

```
# prometheus.yml (datacenter 1)
scrape_configs:
  - job_name: "node"
    static_configs:
      - targets: ["192.168.64.8:9100", "192.168.64.8:9101"]
```

7.3 Step 2 — naive federation (don't do this in production)

```
# prometheus.yml (global) — naive first attempt
scrape_configs:
  - job_name: "prometheus-federation"
    metrics_path: "/federate"
    honor_labels: true
    params:
      'match[]':
        - '{job="node"}'
    static_configs:
      - targets: ["172.16.17.128:9090", "172.16.17.129:9090"]
```

! WHY THIS IS DANGEROUS

- **High cardinality** — sending every instance-level metric to one central server multiplies label combinations and can overwhelm it.
- **Duplicate collisions** — if two sources expose identical metric name + label set (e.g. both say `job="node"`) with the same timestamp but different values, Prometheus treats them as conflicting and drops them.

```
# example dropped-sample error
msg="Error on ingesting samples with different value but same timestamp"
component="scrape manager" scrape_pool=prometheus-federation num_dropped=2
```

7.4 Step 3 — the correct pattern: aggregate + label + federate only that

1 Add a recording rule locally that aggregates instance metrics

Compute sums/rates *by job* instead of shipping every raw series.

2 Attach a distinguishing label (e.g. `datacenter`)

This prevents identical-looking series from different datacenters colliding on the global server.

3 Federate only the aggregated metric names

Use `match[]` with `__name__` filters pointed at your recorded rule names — not raw `job="node"`.

Recording rules (Datacenter 1) — `rules.yaml`:

```
groups:
- name: aggregated-node-metrics
  rules:
    - record: node:cpu_seconds:sum_rate5m
      expr: sum(rate(node_cpu_seconds_total[5m])) by (job)
      labels:
        datacenter: "dc1"
    - record: node:memory_MemAvailable_bytes:sum
      expr: sum(node_memory_MemAvailable_bytes) by (job)
      labels:
        datacenter: "dc1"
```

For Datacenter 2, use the same rules with `datacenter: "dc2"`.

Include the rules file locally:

```
rule_files:
- "rules.yaml"
```

Global Prometheus — federate only the recorded (aggregated) names:

```
scrape_configs:
- job_name: "prometheus-federation"
  metrics_path: "/federate"
  honor_labels: true
  params:
    'match[]':
      - '{__name__="node:cpu_seconds:sum_rate5m"}'
      - '{__name__="node:memory_MemAvailable_bytes:sum"}'
  static_configs:
    - targets: ["172.16.17.128:9090", "172.16.17.129:9090"]
```

Now you can query the global server, filtered by datacenter:

```
node:memory_MemAvailable_bytes:sum{datacenter="dc2", job="node"}
```

REMEMBER `honor_labels: true` keeps the original labels from the local recording rule intact — it stops the global Prometheus from overwriting them.

7.5 Federation do's and don'ts

Action	Recommendation	Note
Federate instance-level metrics	Don't	Avoid bulk <code>node_*</code> per-instance series
Federate aggregated metrics	Do	e.g. <code>node:memory_MemAvailable_bytes:sum</code>
Distinguish origin	Do	Add <code>datacenter: "dc1"</code> as a recording-rule label
Global match filters	Use	Filter by <code>__name__</code> to recorded metric names only
Metrics with an <code>instance</code> label	Don't federate	If you're pulling anything with an <code>instance</code> label, you're doing it wrong

NOT A PROXY Federation is not for copying an entire Prometheus server's contents, and it is not a way to have one Prometheus proxy to another. It exists strictly for pulling **big-picture, aggregate** data upward.

7.6 Alternative: Thanos Query instead of federation

If you want a single pane of glass without restricting yourself to pre-aggregated metrics, Thanos Query can fan a query out across multiple Prometheus servers and merge the results — covered in Section 3.

8. Final Cheat Sheet

Everything, One Page

TSDB blocks

New block every 2h. Contains `chunks`, `index`, `meta.json`, `tombstones`; in-progress data in `wal` / `chunks_head`.

Retention flags

`--storage.tsdb.retention.time=15d` (default)
`--storage.tsdb.retention.size=10G` (default)

Capacity formula

$\text{disk} = \text{retention (sec)} \times \text{samples/sec} \times \text{bytes/sample}$

Remote write / read

`remote_write` = push out. `remote_read` = pull historical back in.

Thanos components

Sidecar (uploads + serves recent) · Store (serves historical from object store) · Query (PromQL fan-out + dedup)

Disable compaction for Thanos

`--storage.tsdb.min-block-duration=2h`
`--storage.tsdb.max-block-duration=2h`

Vertical vs Horizontal scaling

Vertical = different services, different servers.
Horizontal = one service, sharded across servers.

hashmod sharding recipe

- 1) `hashmod` `__address__` by modulus = shard count
- 2) `keep` where result matches this shard's index
- 3) repeat per instance, same target list

Quick load reducers

Widen `scrape_interval` for low-priority jobs.
Set `sample_limit` to fail-fast on runaway scrapes.

Federation — correct pattern

Aggregate locally (recording rule) → label with origin (e.g. `datacenter`) → federate only that metric name.

Federation — never do this

Don't federate raw instance-level metrics. Don't use it as a proxy. Don't pull anything with an `instance` label.

honor_labels

Set `true` on the global scrape job so it doesn't overwrite the original (e.g. `datacenter`) labels from the source.

End-to-end config skeleton

```
# Local Prometheus (per datacenter) — aggregate + label
groups:
- name: aggregated-node-metrics
  rules:
  - record: node_cpu_seconds:sum_rate5m
    expr: sum(rate(node_cpu_seconds_total[5m])) by (job)
    labels:
      datacenter: "dc1"

# Global Prometheus — federate only the aggregate
scrape_configs:
- job_name: "prometheus-federation"
  metrics_path: "/federate"
  honor_labels: true
  params:
    'match[]':
    - '{__name__="node_cpu_seconds:sum_rate5m"}'
  static_configs:
  - targets: ["172.16.17.128:9090", "172.16.17.129:9090"]

# Horizontal shard (per Prometheus instance)
relabel_configs:
- source_labels: [__address__]
  action: hashmod
```

```
modulus: 2
target_label: __tmp_hashmod
- source_labels: [__tmp_hashmod]
  regex: "0"      # change per instance: 0, 1, ... N-1
  action: keep
```

i FINAL RECALL TRICK Picture a company with regional offices. Each **local Prometheus** is a regional office keeping its own detailed paperwork (instance-level metrics). **Federation** is the monthly summary report each office sends to **HQ** (global Prometheus) — never the raw filing cabinets. **Sharding** is splitting one huge office's workload between two desks. **Thanos** is hiring an archivist (object storage) so nobody needs a bigger filing cabinet ever again, plus a receptionist (Query) who can look up any regional office's records from one desk.